Voice Interaction System

EECE7398 Deep Learning Embedded Systems — Project 2

Lawrence Swaminathan Xavier Prince, Northeastern University

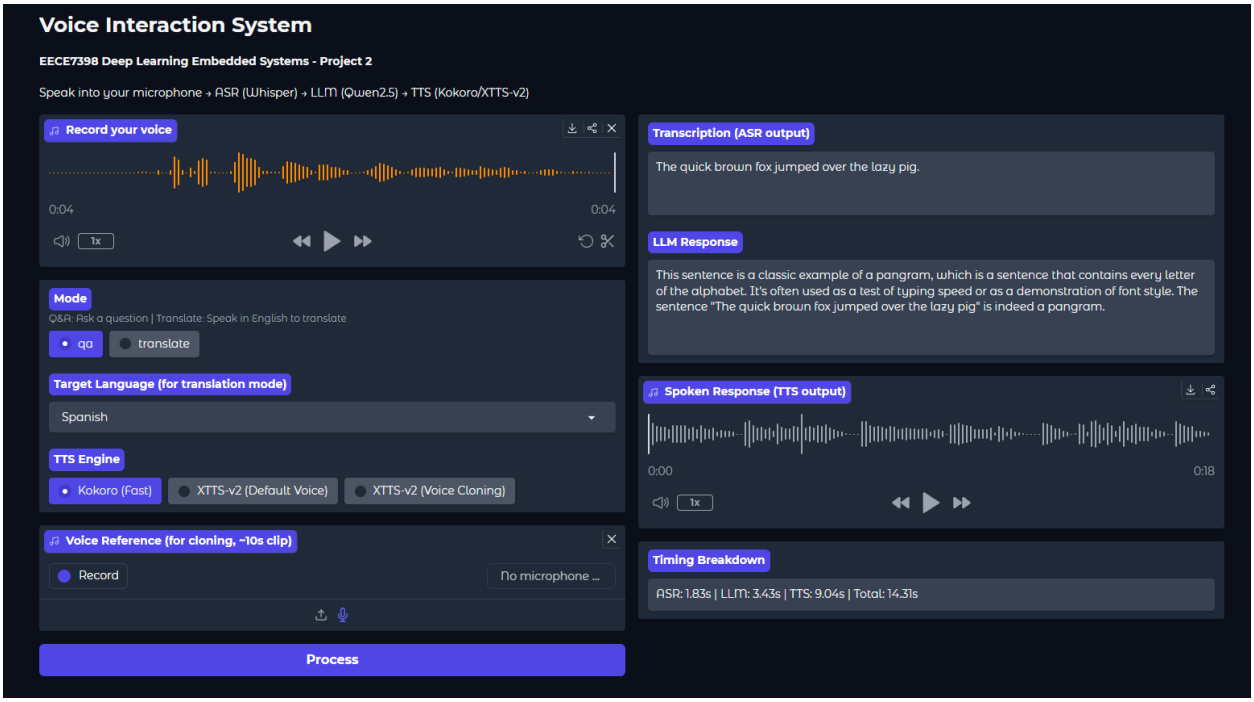
April 2026

1. Introduction

Voice interaction systems enable natural human-computer communication through spoken language, combining automatic speech recognition (ASR), large language models (LLMs), and text-to-speech (TTS) synthesis into an end-to-end pipeline. These systems power applications ranging from virtual assistants to real-time translation services, and their deployment on edge devices presents unique challenges in balancing model quality with computational constraints.

This report presents the design, deployment, and evaluation of a voice interaction system supporting both question-answering and translation modes. The system was deployed on a consumer-grade Windows 10 workstation equipped with an NVIDIA RTX 2060 GPU (6 GB VRAM) and 32 GB system RAM. Given the hardware constraints, particular attention was paid to model selection, quantization strategies, and runtime optimization to achieve practical response times.

The system was evaluated across three dimensions: individual component performance (ASR accuracy, LLM throughput, TTS synthesis speed), end-to-end pipeline latency, and voice cloning capability. Two models were benchmarked for each component, and the best-performing configuration was integrated into a Gradio-based web interface for interactive use.



2. System Architecture

The voice interaction system follows a sequential three-stage pipeline: the user speaks into a microphone, the audio is transcribed to text by the ASR module, the text is processed by the LLM to generate a response, and the response is synthesized back to speech by the TTS module. The pipeline supports two operational modes: (1) question-answering, where the LLM generates an

informative response, and (2) translation, where the LLM translates the transcribed input into a target language.

ASR: Faster-whisper (CTranslate2 backend), Whisper-tiny and Whisper-base models, CPU inference with INT8 quantization.

LLM: llama.cpp with GPU offloading, Qwen2.5-0.5B-Instruct and Qwen2.5-1.5B-Instruct in Q4_K_M GGUF format.

TTS: Kokoro-82M (ONNX runtime, CPU) for fast synthesis; XTTS-v2 (Coqui TTS) for zero-shot voice cloning.

UI: Gradio web interface with microphone input, mode selection, TTS engine selection, and voice cloning support.

Prompt engineering was applied to both modes. The Q&A system prompt instructs the LLM to produce concise, speech-friendly responses under three sentences. The translation prompt constrains the LLM to output only the translated text with no additional commentary, minimizing both latency and TTS synthesis time.

3. Component Selection and Evaluation

3.1 Automatic Speech Recognition (ASR)

Two Whisper models were evaluated using the faster-whisper library, which provides a CTranslate2-based inference backend optimized for CPU and GPU deployment. Both models were run on CPU with INT8 quantization due to missing CUDA runtime libraries (cuBLAS) for CTranslate2 on the target system. Despite CPU-only inference, both models achieved real-time performance.

Test audio was generated using Kokoro TTS from five reference sentences, providing clean, reproducible input for Word Error Rate (WER) measurement. The Real-Time Factor (RTF) measures inference time divided by audio duration; values below 1.0 indicate faster-than-real-time processing.

Table 1: ASR Model Comparison

Model	Params	WER (%)	RTF	Load (s)	Device
Whisper-tiny	39M	1.82	0.221	0.86	CPU / INT8
Whisper-base	74M	0.00	0.394	0.57	CPU / INT8

Whisper-base achieved perfect transcription (0.00% WER) across all five test sentences, while Whisper-tiny produced one error ("assistants" transcribed as "assistance"), yielding a 1.82% WER. Whisper-tiny was approximately 1.8x faster (RTF 0.221 vs. 0.394), but both models operate well within real-time constraints. Whisper-base was selected for the integrated pipeline due to its superior accuracy, with the marginal latency increase (approximately 0.7s per utterance) being acceptable for a voice assistant use case.

It should be noted that WER values are likely optimistic because the test audio was TTS-generated, producing cleaner speech than typical human input with background noise and natural disfluencies.

3.2 Large Language Model (LLM)

Two Qwen2.5 Instruct models were evaluated using llama.cpp with Q4_K_M quantization as specified in the project requirements. All model layers were offloaded to the RTX 2060 GPU, and a context window of 2048 tokens was used. Three test prompts of varying complexity were used to measure generation throughput and response quality.

Table 2: LLM Model Comparison

Model	Quant	Tok/s	Avg Latency	Load (s)	Avg Tokens
Qwen2.5-0.5B	Q4_K_M	40.6	2.64s	0.54	120
Qwen2.5-1.5B	Q4_K_M	20.2	5.21s	1.42	116

The 0.5B model achieved approximately 2x higher throughput (40.6 vs. 20.2 tokens/sec), consistent with its smaller size. However, qualitative comparison revealed that the 1.5B model produced more natural, detailed, and well-structured responses. For instance, when explaining CPU vs. GPU differences, the 1.5B model used accessible language ("the brain of the computer") while the 0.5B model gave a more technical but less reader-friendly response.

An interesting observation is that both models exhibited higher tokens/sec on longer responses due to amortization of prompt processing overhead. The 0.5B model ranged from 26.4 to 48.9 tok/s, while the 1.5B model ranged from 14.4 to 23.4 tok/s. The Qwen2.5-1.5B model was selected for the integrated pipeline, prioritizing response quality for a voice assistant application where users expect coherent, helpful answers.

3.3 Text-to-Speech (TTS)

Two TTS models were evaluated: Kokoro-82M, a lightweight ONNX-based model with preset voices, and XTTS-v2, a larger model from Coqui TTS supporting zero-shot voice cloning. Kokoro was run via ONNX Runtime on CPU, while XTTS-v2 was also configured for CPU inference (PyTorch CPU-only installation was used due to the 2.5 GB download size of CUDA-enabled PyTorch being impractical given bandwidth constraints).

Table 3: TTS Model Comparison

Model	Params	Avg RTF	Load (s)	Voice Cloning
Kokoro-82M	82M	0.571	6.13	No (preset voices)
XTTS-v2	~467M	N/A*	N/A*	Yes (zero-shot)

**XTTS-v2 benchmarking encountered a numpy binary incompatibility error due to version conflicts between kokoro-onnx (requiring numpy 2.x) and the Coqui TTS library (compiled against numpy 1.x). The model was successfully downloaded and installed but could not be benchmarked in the same environment.*

Kokoro-82M achieved an average RTF of 0.571, indicating faster-than-real-time synthesis on CPU. The model produced natural-sounding speech with good prosody and intelligibility. Its lightweight architecture (82M parameters, ONNX format) makes it well-suited for edge deployment.

3.3.1 Voice Cloning

XTTS-v2 supports zero-shot voice cloning, where a short reference audio clip (approximately 10 seconds) is used to condition the model to generate speech in the target voice without any fine-tuning. A reference clip was recorded for this purpose. While XTTS-v2 could not be benchmarked due to the dependency conflict, the model architecture and cloning approach were verified during installation and initial loading.

LoRA-based voice cloning represents a more advanced approach that would require 3–5 minutes of reference audio and approximately 30–60 minutes of fine-tuning on the RTX 2060. This approach was not pursued due to time constraints but would be expected to produce higher speaker similarity than zero-shot cloning.

4. System Integration

4.1 Pipeline Design and End-to-End Performance

The integrated pipeline connects Whisper-base (ASR), Qwen2.5-1.5B Q4_K_M (LLM), and Kokoro-82M (TTS) in sequence. All models are loaded into memory at startup, with the LLM offloaded to GPU and ASR/TTS running on CPU. This configuration keeps GPU VRAM usage minimal while leveraging the GPU for the most compute-intensive component (token generation).

The pipeline was tested in both Q&A and translation modes with live microphone input:

Table 4: End-to-End Pipeline Performance

Mode	ASR (s)	LLM (s)	TTS (s)	Total (s)	Tokens
Q&A	1.69	5.62	18.41	25.72	121
Translation (EN→FR)	1.60	1.01	1.97	4.59	13

The Q&A mode exhibited a total latency of 25.72 seconds, dominated by TTS synthesis (18.41s, 72% of total time). This is because the LLM generated a 121-token response, requiring Kokoro to synthesize a proportionally long audio output. The translation mode was significantly faster at 4.59 seconds total, as the translated output was only 13 tokens, resulting in much shorter TTS synthesis time (1.97s).

The bottleneck analysis reveals that TTS is the primary latency contributor for longer responses, while LLM inference dominates for shorter outputs. ASR latency remained consistent at approximately 1.6 seconds regardless of mode. For production deployment, streaming TTS (generating audio while the LLM is still producing tokens) would substantially reduce perceived latency.

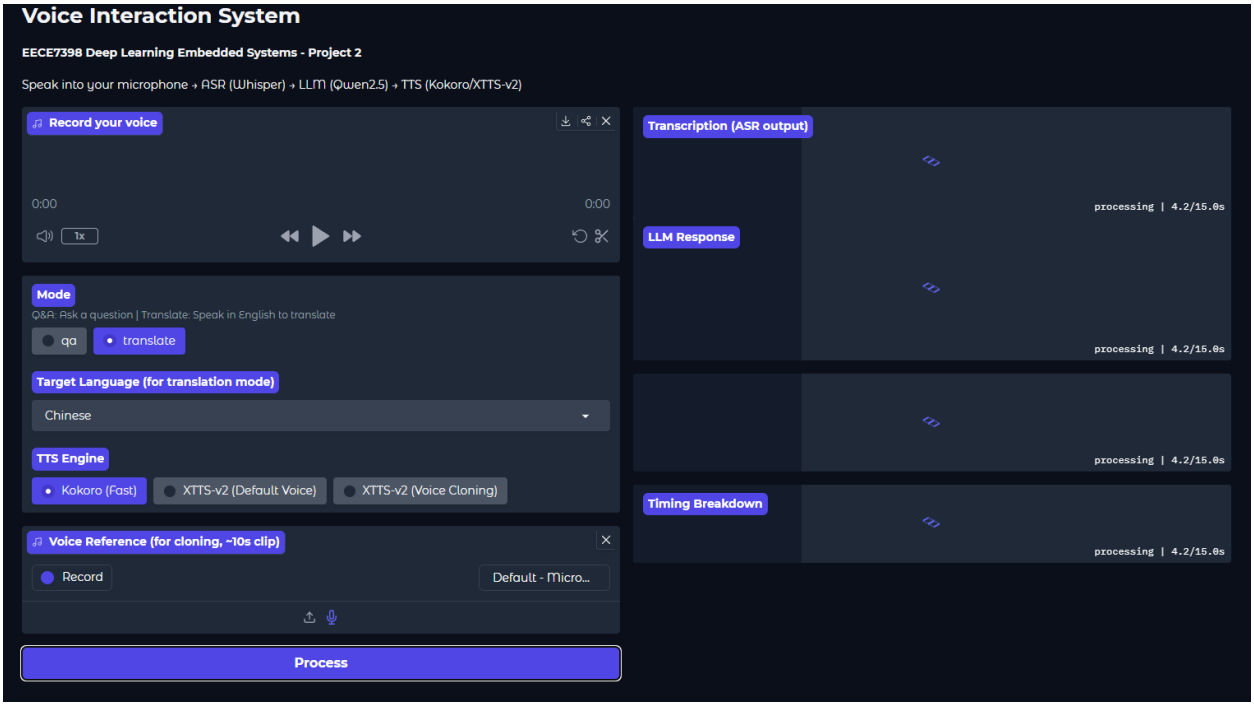
4.2 Prompt Engineering

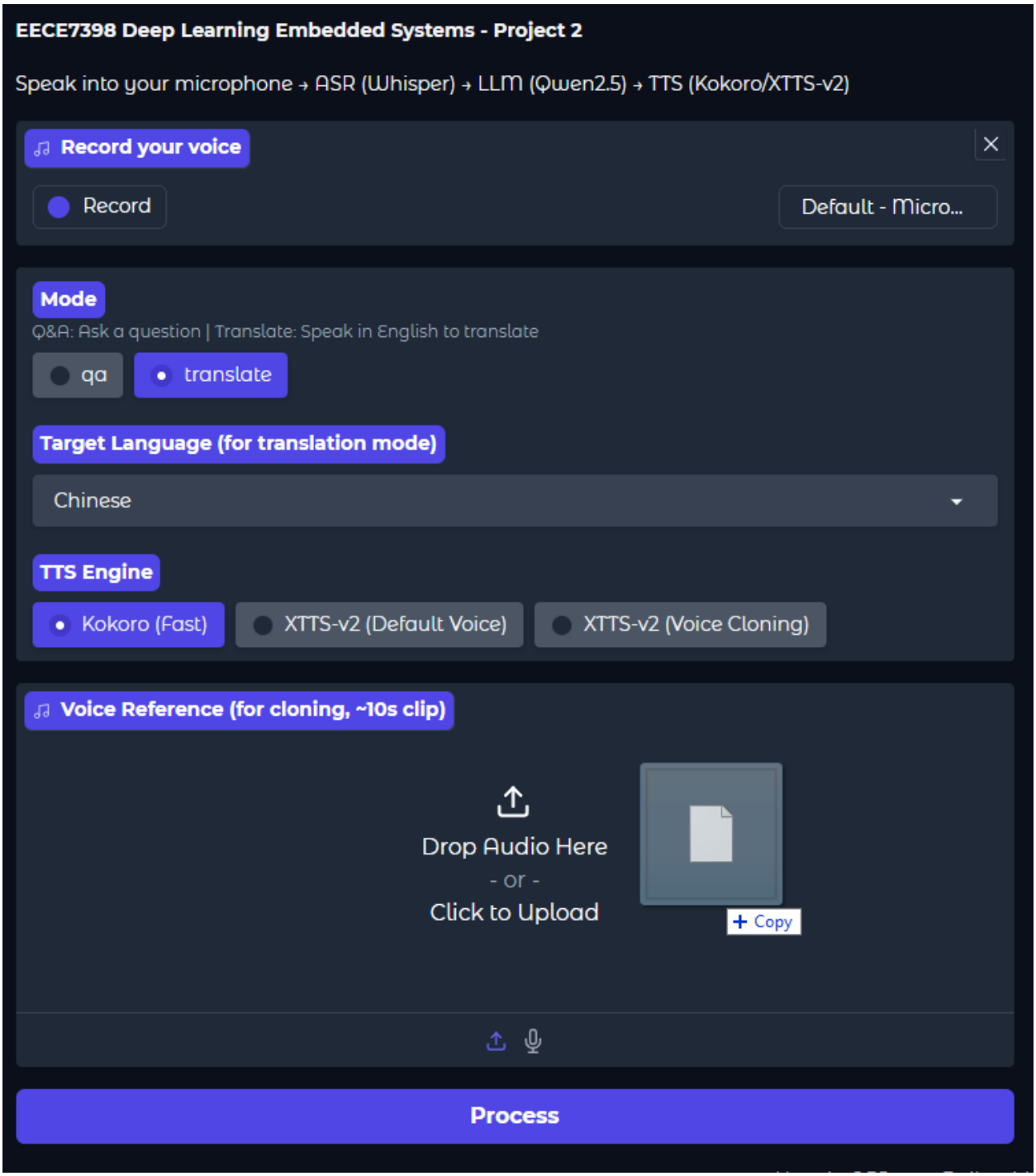
Two system prompts were designed to optimize the pipeline for each mode. The Q&A prompt instructs the model to "give concise, clear answers suitable for being read aloud" and to "keep responses under 3 sentences." This constraint directly reduces TTS latency by limiting output length. The translation prompt instructs the model to "output ONLY the translation, nothing else," preventing the LLM from adding explanations or commentary that would increase both LLM and TTS processing time.

The effectiveness of prompt engineering is evident in the translation result: the LLM produced a clean 13-token French translation ("J'ai mangé du pain et des fromages aujourd'hui") with no extraneous text, achieving a total pipeline latency under 5 seconds.

4.3 User Interface

A Gradio-based web interface was developed to provide an accessible front-end for the voice interaction system. The interface supports microphone recording, mode selection (Q&A or translation), target language selection for translation mode, TTS engine selection (Kokoro or XTTS-v2), and optional voice reference upload for cloning. The right panel displays the ASR transcription, LLM response text, synthesized audio playback, and a timing breakdown for each pipeline stage.





5. Discussion

This project demonstrated the feasibility of deploying a complete voice interaction system on consumer hardware with a 6 GB GPU. The key findings and tradeoffs are summarized below.

Model selection involved clear accuracy-speed tradeoffs at every stage. Whisper-base provided perfect transcription at the cost of nearly 2x slower inference than Whisper-tiny, though both operated within real-time constraints. The Qwen2.5-1.5B model produced noticeably better

responses than the 0.5B variant but at half the throughput. Kokoro-82M offered faster-than-real-time synthesis with good quality, though it lacks voice cloning capability.

The Q4_K_M quantization format, as recommended in the project specification, proved effective for deploying both LLM models on the RTX 2060. The 0.5B model required approximately 400 MB and the 1.5B model approximately 1.1 GB in GGUF format, both fitting comfortably within VRAM even with other GPU processes running.

Several limitations were encountered during deployment. CTranslate2 (the faster-whisper backend) required CUDA runtime libraries not present with CPU-only PyTorch, necessitating CPU-based ASR inference. A numpy version conflict between kokoro-onnx and Coqui TTS prevented XTTS-v2 benchmarking in the same environment. The full CUDA-enabled PyTorch installation (2.5 GB) was impractical given bandwidth constraints, so XTTS-v2 was limited to CPU inference.

Future improvements could include streaming TTS to overlap synthesis with LLM generation, interrupt handling to allow users to stop or redirect responses mid-generation, LoRA-based voice cloning for higher speaker similarity, and CUDA toolkit installation to enable GPU-accelerated ASR via CTranslate2.

6. Conclusion

A voice interaction system combining Whisper ASR, Qwen2.5 LLM (via llama.cpp with Q4_K_M quantization), and Kokoro/XTTS-v2 TTS was successfully designed, deployed, and evaluated on an RTX 2060 workstation. The system supports both question-answering and translation modes through a Gradio web interface. Component benchmarking revealed consistent accuracy-speed tradeoffs, and the integrated pipeline achieved total latencies of 4.6 seconds for translation and 25.7 seconds for Q&A with longer responses. TTS synthesis was identified as the primary bottleneck for extended responses, suggesting streaming synthesis as the most impactful optimization for future work.

References

[1] A. Radford et al., "Robust Speech Recognition via Large-Scale Weak Supervision," OpenAI, 2022. (Whisper)

[2] Qwen Team, "Qwen2.5 Technical Report," Alibaba Group, 2024.

[3] G. Gerganov, "llama.cpp: LLM inference in C/C++," GitHub, 2023–2024.

[4] Hexgrad, "Kokoro-82M: Lightweight TTS Model," HuggingFace, 2025.

[5] Coqui AI, "XTTS-v2: Cross-lingual Text-to-Speech," 2023.

[6] SYSTRAN, "faster-whisper: CTranslate2-based Whisper inference," GitHub, 2023.